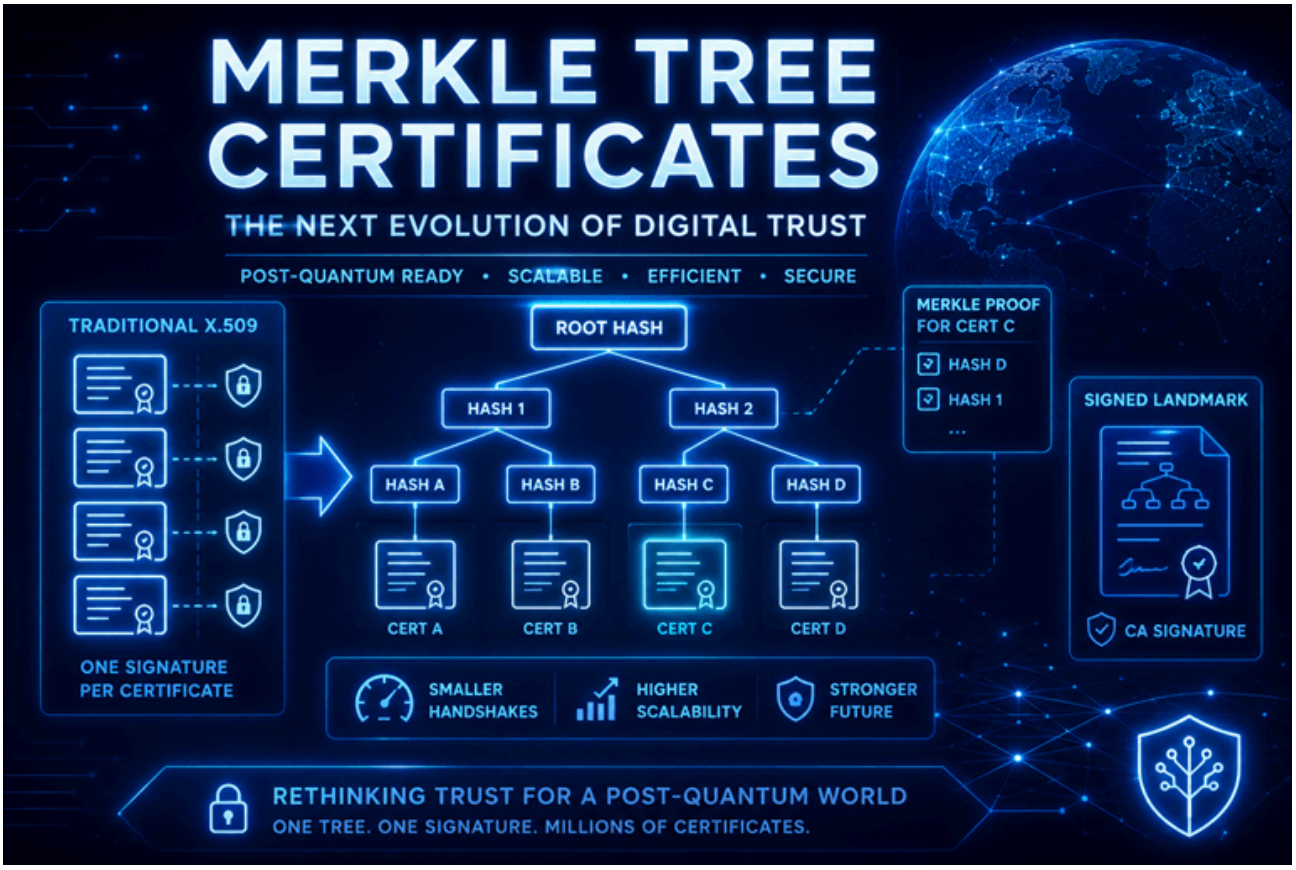
How Merkle Tree Certificates Work – Michael Waterman

michaelwaterman.nl/2026/06/27/how-merkle-tree-certificates-work

Michael Waterman June 27, 2026



For more than 35 years, X.509 certificates have formed the foundation of digital trust on the Internet and in many organizations alike. From HTTPS and VPNs to enterprise authentication and code signing, the basic architecture has remained unchanged. As we enter the post-quantum era, however, it may not be the cryptographic algorithms that need the biggest overhaul, but the certificate model itself.

Imagine a Certificate Authority such as Let's Encrypt issuing hundreds of millions of post-quantum certificates. Every one of those certificates now carries a digital signature that is significantly larger than today's RSA or ECC signatures. Suddenly, TLS handshakes become larger, certificate chains consume more bandwidth, Certificate Transparency logs grow faster, and browsers have considerably more data to process. Everything just slows down.

The challenge is no longer whether post-quantum cryptography works. The challenge is whether the X.509 certificate architecture can continue to scale in a post-quantum world. This is exactly the problem that Merkle Tree Certificates (MTC) are designed to solve.

Rather than signing every certificate individually, Merkle Tree Certificates introduce an entirely different approach to digital trust. By replacing millions of individual signatures with

a cryptographically verifiable Merkle Tree, they dramatically reduce the amount of data that needs to be signed, transmitted, and verified. The result is a certificate architecture that is far better suited for Internet-scale deployment of post-quantum cryptography.

In this article, I'll explore what Merkle Tree Certificates are, why they were introduced, how they work, and why they may become one of the most significant changes to PKI since the introduction of X.509 itself. We'll at least that's my opinion, let's dive in!

And why X.509 now becomes a problem?

For decades, X.509 has proven to be an incredibly successful certificate format. It is simple, flexible, interoperable, and supported by virtually every operating system, browser, and network appliance in existence. So why change something that has worked so well for more than three decades? The answer is simple: X.509 was never designed for the scale and cryptographic requirements of today's Internet. When the X.509 standard was first introduced, Certificate Authorities issued relatively small numbers of certificates using algorithms with compact signatures. Every certificate was individually signed by a Certification Authority, and validating that signature was computationally inexpensive. Even as the Internet grew, this model continued to scale remarkably well. Post-quantum cryptography changes that assumption.

Unlike RSA or Elliptic Curve Cryptography (ECC), post-quantum signature algorithms produce significantly larger public keys and digital signatures. While this is a necessary trade-off to achieve quantum resistance, it also has consequences that extend far beyond the certificate itself. Every byte added to a certificate eventually finds its way into the TLS handshake. Larger certificates result in larger certificate chains, increased network bandwidth, more memory usage, larger Certificate Transparency logs, and additional processing for both servers and clients. None of these factors are particularly problematic on their own, but together they become significant at Internet scale.

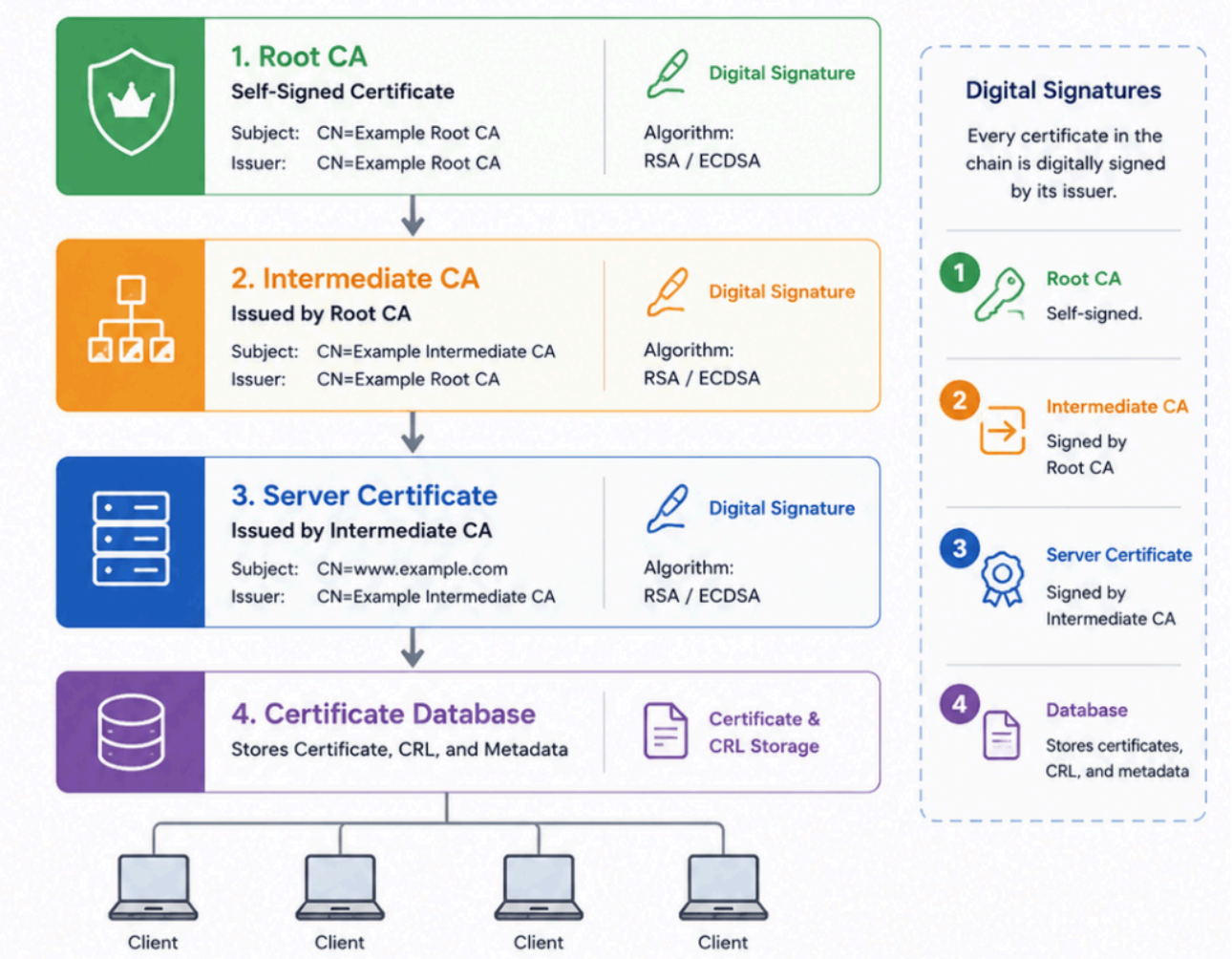
Now consider a public Certificate Authority issuing hundreds of millions of certificates every year. Every certificate requires its own digital signature. Every client validates that signature independently. Every certificate is logged individually. Every TLS handshake carries another copy of that signature across the network. Suddenly, the problem is no longer the cryptographic algorithm itself. The real question becomes:

Does every certificate really need its own digital signature?

That single question fundamentally changes how we think about digital certificates. Instead of searching for smaller post-quantum signatures, researchers began asking a different question entirely, what if millions of certificates could share a single cryptographic signature while remaining individually verifiable? That idea ultimately led to the development of Merkle Tree Certificates.

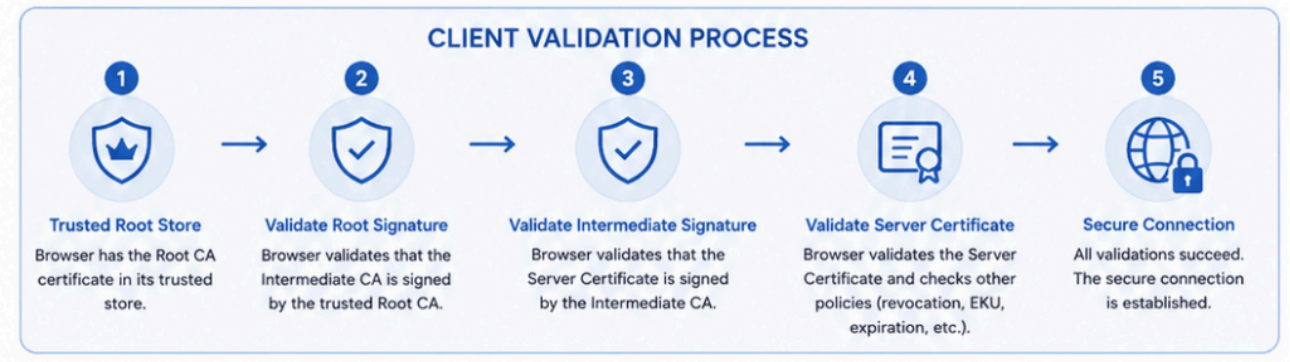
A quick refresher on X.509

Before I dive into Merkle Tree Certificates, it's worth a quick recap at how trust is established in today's Public Key Infrastructure. Whenever you visit a secure website, your browser receives a certificate chain that typically consists of three certificates:



X.509 Certificate Chain

Each certificate in the chain contains a digital signature created by its issuer. The Root CA is trusted because its certificate already exists in your operating system or browser's trusted root store. The Intermediate CA is trusted because it has been digitally signed by the Root CA. Finally, the server certificate is trusted because it has been digitally signed by the Intermediate CA. During the TLS handshake, your browser validates every signature in the chain before deciding whether the connection can be trusted. The process looks something like this:



Certificate validation process

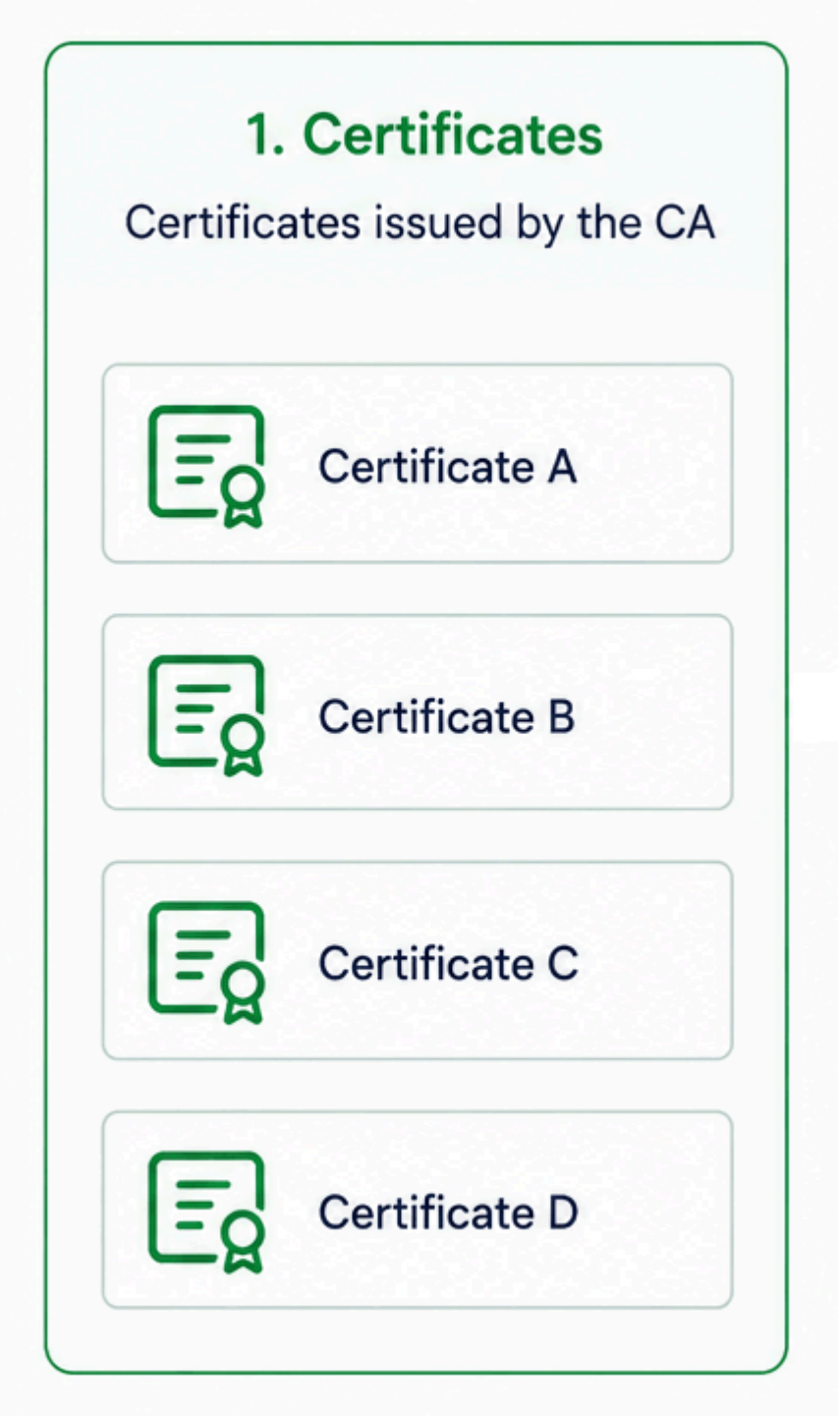
Note! Key takeaway is that every certificate in the chain requires it's own digital signature. Every client must validate every signature, every time.

This model has served us exceptionally well for decades. It is simple, well understood, and highly secure. However, there is one important characteristic that often goes unnoticed. Every certificate carries its own individual digital signature. Whether a Certificate Authority issues one certificate or one hundred million certificates, every single certificate must be signed individually, and those are expensive operations. Likewise, every client that receives that certificate must validate that individual signature before establishing trust. For traditional cryptographic algorithms such as RSA and ECC, this has never been a significant problem. Digital signatures are relatively compact, computationally efficient, and have scaled remarkably well for decades.

As I'll show you in the next chapter, post-quantum cryptography changes that equation, not because the signatures become less secure, but because they become significantly larger. Once certificates are issued at Internet scale, the assumption that every certificate requires its own signature starts to become increasingly expensive.

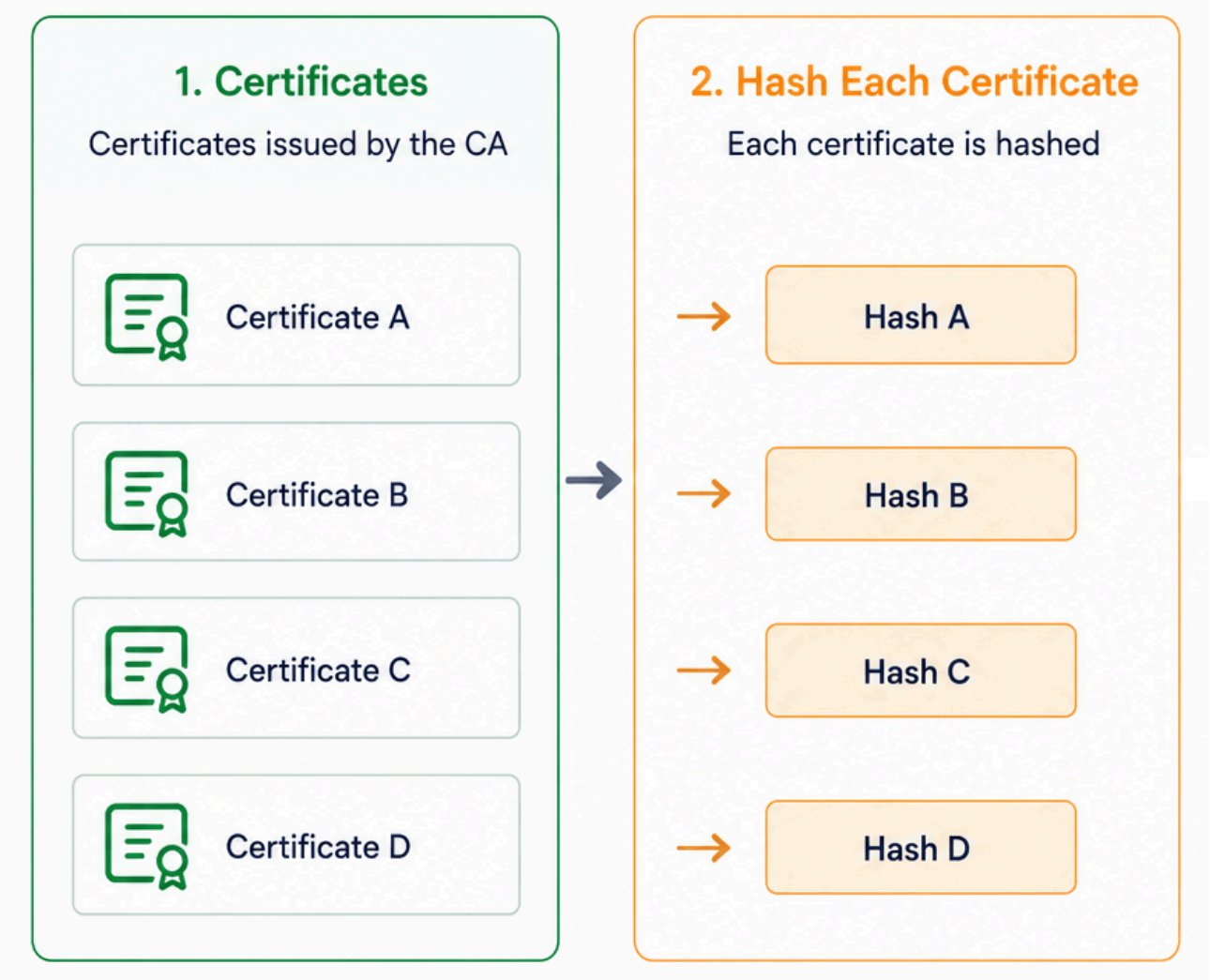
A different way of thinking

At this point, I've identified the fundamental limitation of the traditional X.509 model: every certificate requires its own digital signature. So let's ask a different question: " *What if a Certificate Authority didn't have to sign every certificate individually?*". Imagine a CA issues four certificates.



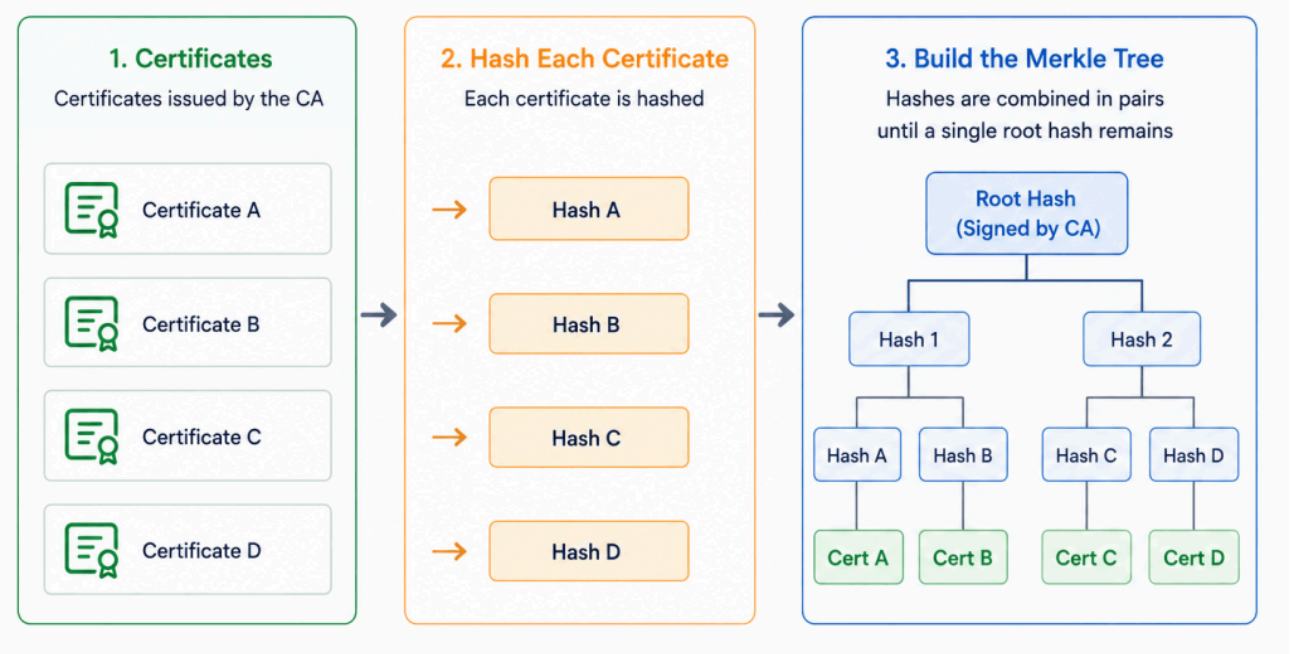
As an example, image four certificates

Instead of immediately signing each certificate, the CA first creates a cryptographic hash of every certificate. Remember, creating a hash is significantly cheaper than a signing operation.



Hash creation

A cryptographic hash is simply a unique digital fingerprint of the certificate. Even changing a single bit in the certificate produces a completely different hash, making it an ideal way to detect any modification. Next, the hashes are combined in pairs and hashed again. Finally, those two hashes are combined one last time to produce a single value known as the "*Root Hash*".



Merkle tree explained

And this entire structure is what we call a *Merkle Tree*. Now comes the important part, pay close attention. Instead of signing every certificate individually, the Certificate Authority signs only the "*Root Hash*" of the Merkle Tree. That single signature now represents every certificate contained within the tree. The certificates themselves are no longer individually signed. Instead, each certificate is accompanied by a small "*Merkle Proof*", which allows a client to mathematically prove that the certificate belongs to the signed tree. We'll look at how that proof works in the next chapter. For now, the key takeaway is simple:

Instead of creating one digital signature for every certificate, the CA creates one digital signature for an entire collection of certificates.

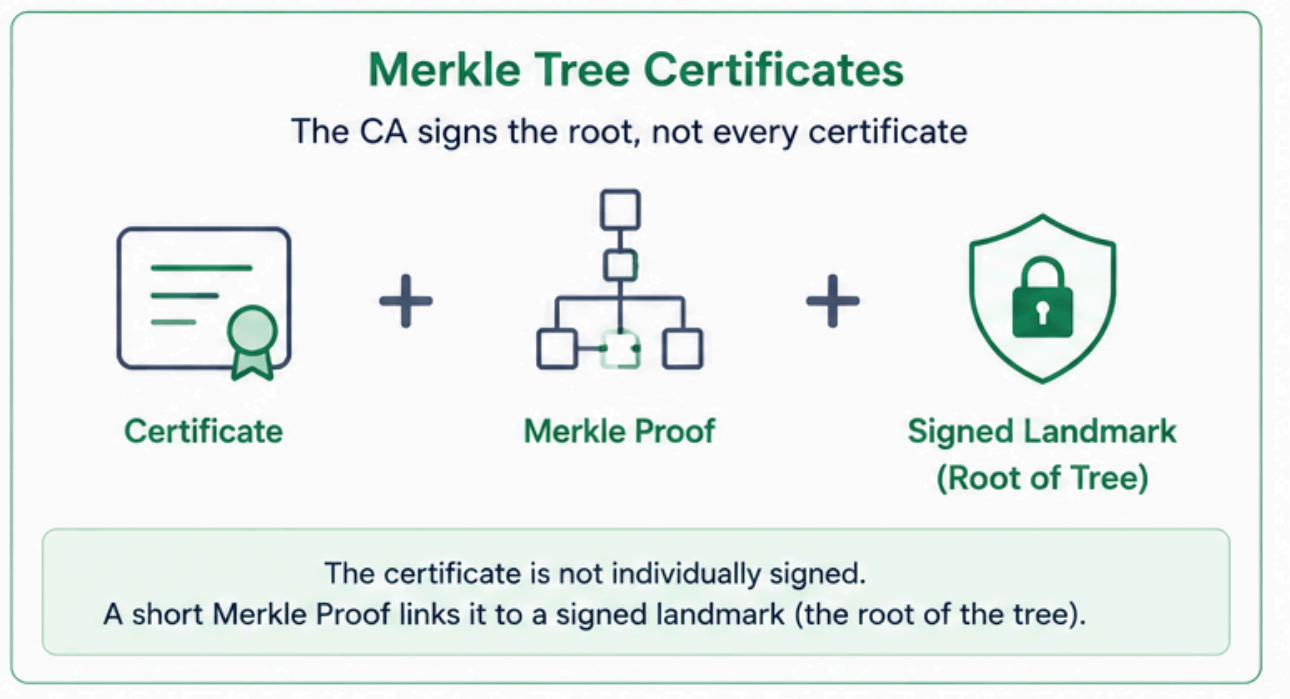
That single design change is what makes Merkle Tree Certificates fundamentally different, and a whole lot faster from traditional X.509 certificates.

What actually changes?

At first glance, Merkle Tree Certificates may seem like an entirely new type of certificate. In reality, the biggest change is surprisingly small. The certificate itself doesn't fundamentally change. What changes is, *how trust is attached to that certificate.* In the traditional X.509 model, every certificate contains its own digital signature created by the issuing Certificate Authority. Conceptually, a certificate looks like this:

Signed certificate

That signature is unique to the certificate. Every browser, operating system, or TLS client validates that signature before deciding whether the certificate can be trusted. Merkle Tree Certificates take a different approach. Instead of carrying an individual CA signature, every certificate is accompanied by a small Merkle Proof. That proof demonstrates that the certificate belongs to a Merkle Tree whose Root Hash has been digitally signed by the Certificate Authority. Let’s Encrypt refers to this signed root as a Landmark. The model therefore becomes:

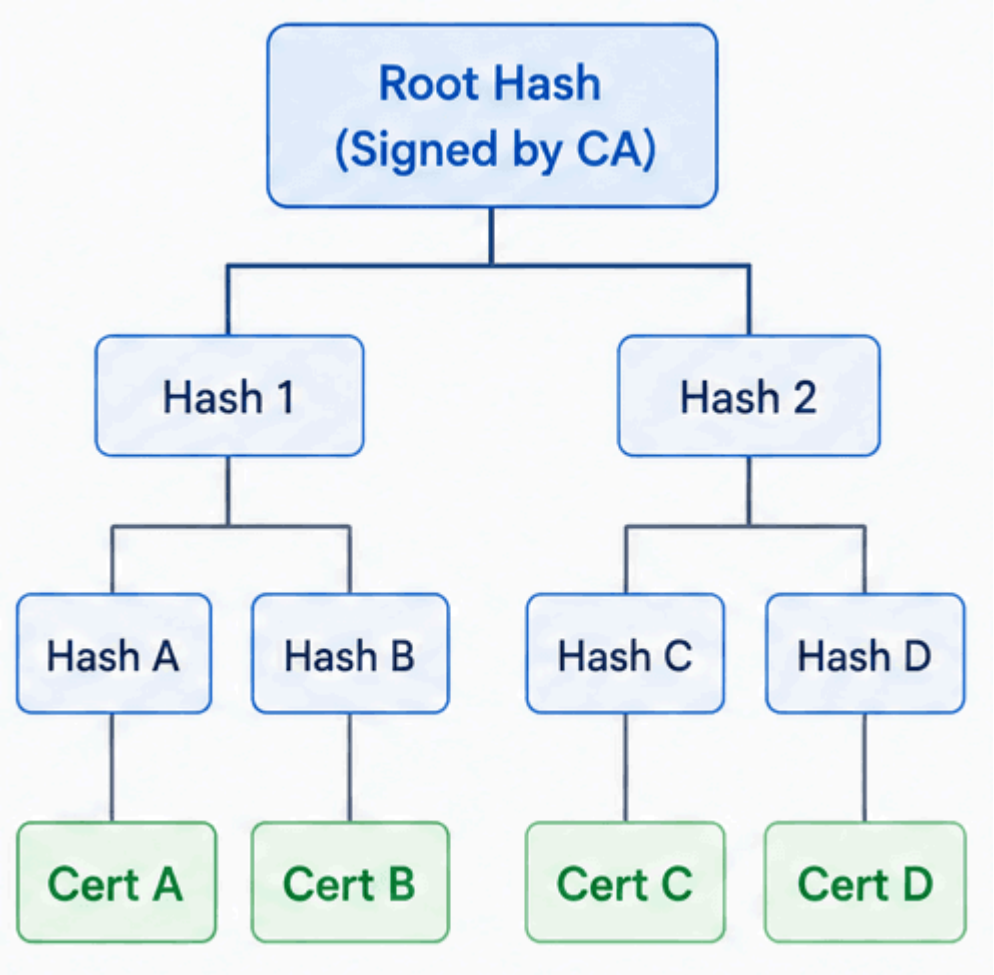


Merkle proof & the signed Landmark

The certificate itself is no longer individually signed. Instead, trust is derived from the signed Landmark together with the Merkle Proof, which mathematically demonstrates that the certificate is part of the signed tree. This is the fundamental shift introduced by Merkle Tree Certificates. Instead of attaching trust directly to every individual certificate, trust is attached to an entire collection of certificates. That single design decision dramatically reduces the number of digital signatures that need to be created, transmitted, and verified, while preserving the same cryptographic level of trust. At this point, one obvious question remains. If the Certificate Authority only signs the Landmark, how can a browser prove that an individual certificate really belongs to that signed Merkle Tree? The answer lies in the Merkle Proof.

How does a browser know the certificate Is genuine?

You might be thinking, so, If the Certificate Authority no longer signs every certificate individually, how can a browser determine whether a certificate is genuine? Glad you asked! The answer lies in something called a *Merkle Proof*.Let’s use the same Merkle Tree from the previous chapter.



Merkle tree

Imagine a web server presents *Certificate C* during a TLS handshake. The browser does *not*receive an individual CA signature for that certificate. Instead, it receives three pieces of information:

- The certificate itself
- A Merkle Proof
- The signed Landmark

The first step is straightforward. The browser calculates the cryptographic hash of certificate C. The Merkle Proof then provides only the additional hashes required to reconstruct the Root Hash. In this example, the proof contains:

- Hash D
- Hash 1

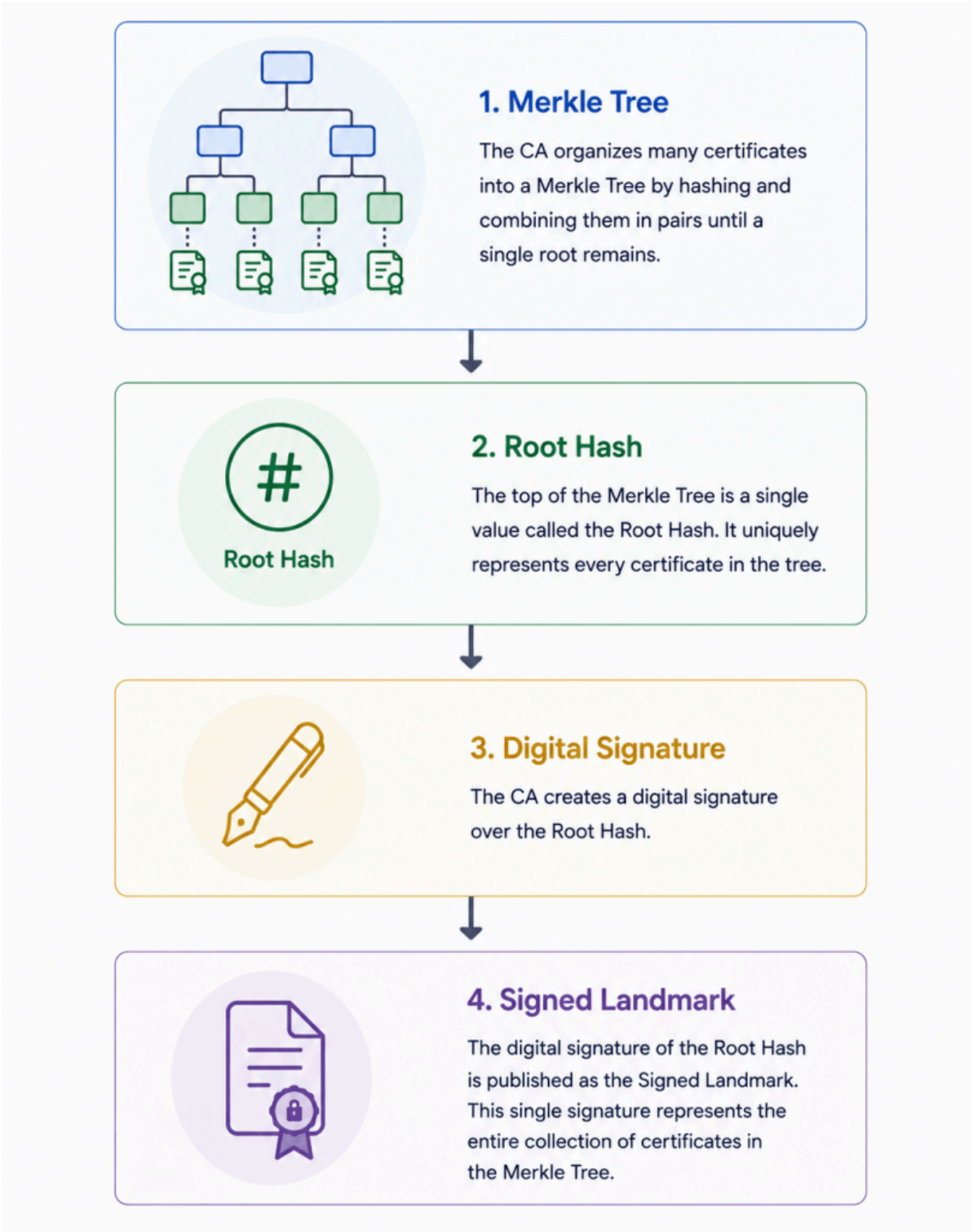
Using those values, the browser rebuilds the tree from the bottom up.

Calculation of the Merkle tree

If the calculated Root Hash matches the Root Hash contained in the signed Landmark, the browser has mathematically proven that Certificate C belongs to the signed Merkle Tree. This process is remarkably efficient. The browser never needs access to every certificate in the tree. It only requires the certificate itself and a very small Merkle Proof containing the sibling hashes needed to reconstruct the Root Hash.

What Is a signed landmark?

The final piece of the puzzle is the *Signed Landmark*. A Landmark is simply the Certificate Authority's digital signature over the Root Hash of a Merkle Tree. In the traditional X.509 model, a Certificate Authority creates a digital signature for every certificate it issues. With Merkle Tree Certificates, the CA creates a single digital signature that represents an entire collection of certificates. Instead of this signing all certificates individually.The Certificate Authority now signs only this:



Signed landmark

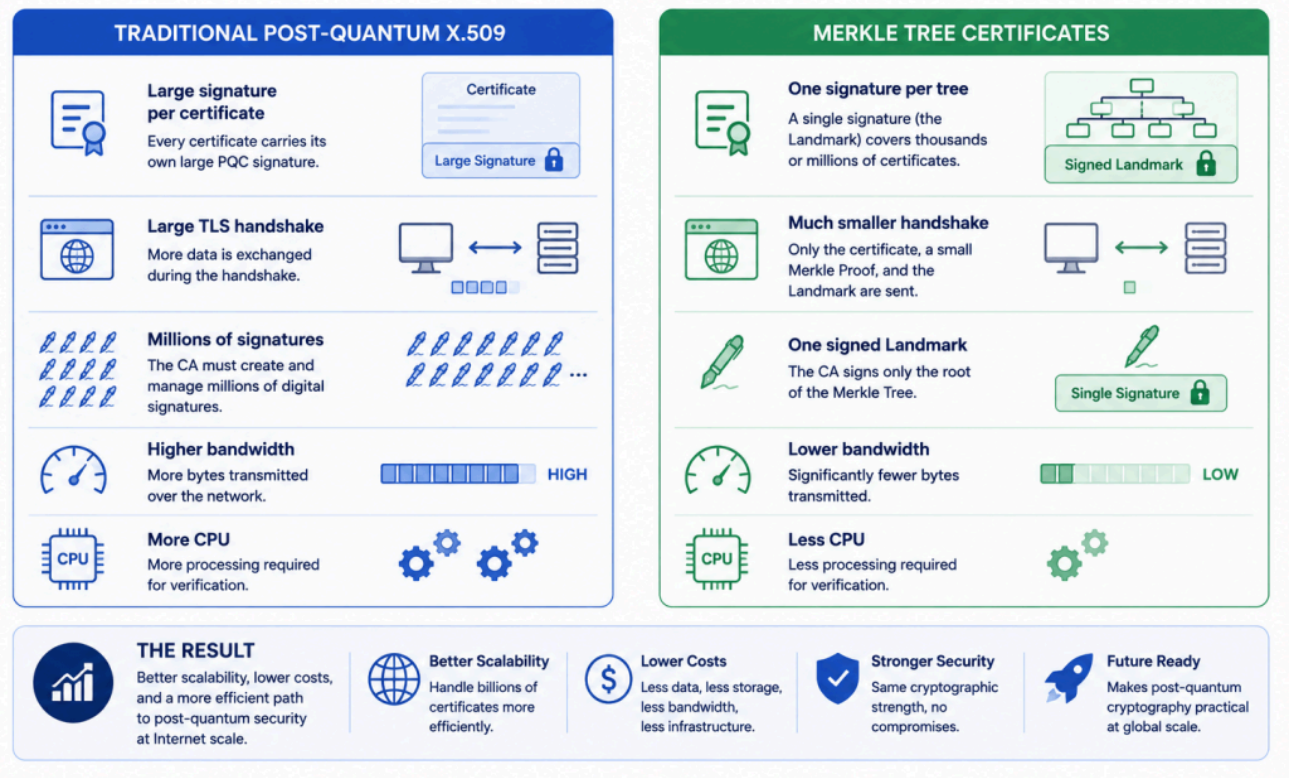
Every certificate contained within that tree ultimately derives its trust from that one signed Landmark. This is the key innovation behind Merkle Tree Certificates. The browser no longer verifies an individual certificate signature. Instead, it verifies that the certificate belongs to a collection whose Root Hash has been signed by a trusted Certificate Authority. In other words, trust has moved from the individual certificate to the Merkle Tree itself.

The Impact of Merkle Tree Certificates

By now, I've shown you that Merkle Tree Certificates don't introduce a new cryptographic algorithm. Instead, they introduce a fundamentally different way of establishing trust. That architectural change has significant practical benefits, particularly as the industry transitions towards post-quantum cryptography. The comparison below summarizes the most important differences.

Traditional X.509	Merkle Tree Certificates
Every certificate carries its own CA signature	Thousands or millions of certificates share a single signed Landmark
Large post-quantum signatures are transmitted with every certificate	Only a compact Merkle Proof accompanies each certificate
Larger TLS handshakes	Significantly smaller TLS handshakes
More bandwidth consumption	Reduced network traffic
More signature generation and verification	Fewer digital signature operations
Scales well today	Designed to scale for Internet-scale post-quantum PKI

These improvements become increasingly important as post-quantum signature algorithms grow considerably larger than today's RSA and ECC signatures. Rather than attempting to reduce the size of the signatures themselves, Merkle Tree Certificates reduce the number of signatures that need to exist in the first place.



Overview of Merkle Tree advantages

Rethinking certificate revocation

One particularly benefit of this architecture is its impact on certificate revocation. Today's PKI relies heavily on Certificate Revocation Lists (CRLs) and the Online Certificate Status Protocol (OCSP) to determine whether a certificate should still be trusted. Maintaining that infrastructure introduces additional complexity, operational overhead, and privacy considerations. Merkle Tree Certificates make it practical to move towards much shorter certificate lifetimes. Instead of issuing certificates that remain valid for months, Certificate Authorities can issue certificates that expire after days, or potentially even hours.

When certificates naturally expire within a very short period of time, the need for traditional revocation mechanisms is greatly reduced. In many scenarios, simply replacing an expired certificate becomes more efficient than maintaining complex revocation infrastructure. This isn't a completely new idea. The Web PKI has already been moving towards shorter certificate lifetimes for several years. Merkle Tree Certificates simply make that transition far more practical in a post-quantum world, where the cost of transmitting large signatures becomes increasingly significant. Ultimately, Merkle Tree Certificates are not just about making certificates smaller.

They represent a shift towards a PKI architecture that is designed for Internet scale, where billions of certificates can be issued, validated, and replaced efficiently without sacrificing cryptographic security.

What about Microsoft Active Directory Certificate Services?

If you've been following Microsoft's recent investments in post-quantum cryptography, you may be wondering whether Merkle Tree Certificates will eventually become part of Active Directory Certificate Services (AD CS). At the time of writing, the answer is simple: we don't know. Microsoft has made significant progress in preparing the Windows PKI ecosystem for the post-quantum era. Releases from May 2026 have introduced support for post-quantum cryptographic signing algorithms, with additional capabilities already announced for future Windows Server updates. It's clear that Microsoft's PKI roadmap is still evolving, and we can expect more innovations as post-quantum standards continue to mature.

Interestingly, during a recent Microsoft webinar covering the future of Windows PKI and Active Directory Certificate Services, Merkle Tree Certificates were not discussed. That doesn't mean Microsoft isn't evaluating the technology, it simply means there has been no public announcement regarding MTC support in Windows or AD CS, huge difference. It's also worth remembering that Microsoft's enterprise PKI ecosystem has different design goals than the public Web PKI. Public Certificate Authorities issue hundreds of millions of certificates every year, making scalability one of their biggest challenges. Enterprise PKI environments often operate at a much smaller scale, meaning the business case for adopting Merkle Tree Certificates may be different.

That said, post-quantum cryptography is still a rapidly evolving field. New standards, implementations, and architectural approaches continue to emerge across the industry. Whether Merkle Tree Certificates eventually become part of Microsoft's certificate infrastructure remains to be seen, but it's certainly a technology worth watching. If you're responsible for Microsoft PKI or Active Directory Certificate Services, my advice is simple, keep an eye on the Microsoft Learn documentation, Windows Server release notes, and future announcements from the Windows PKI team. The journey towards a practical post-quantum PKI is far from over, and I suspect we've only seen the beginning.

Final thoughts

For more than 35 years, X.509 certificates have served as the foundation of digital trust. They have enabled secure websites, enterprise authentication, software signing, VPNs, smart cards, and countless other technologies that we rely on every day. The arrival of post-quantum cryptography doesn't mean that X.509 has suddenly become obsolete. On the contrary, it has proven to be one of the most successful and resilient standards ever developed.

However, the post-quantum era forces us to think beyond simply replacing RSA or ECC with larger, quantum-resistant algorithms. As certificate sizes grow and Internet-scale PKI continues to expand, the architecture itself becomes just as important as the cryptography.

That’s exactly what makes Merkle Tree Certificates so fascinating. It doesn’t replace the mathematics behind digital signatures, it rethinks how trust is established in the first place. Whether Merkle Tree Certificates eventually become the next generation of Web PKI remains to be seen. Like every major change in cryptography, adoption will take time, standards will continue to evolve, and software vendors will gradually introduce support where it makes sense.

For those of us working in PKI, however, this is an exciting time. We’re witnessing one of the most significant shifts in certificate architecture since the introduction of X.509 itself. Even if Merkle Tree Certificates never become part of every PKI implementation, the ideas behind them are already influencing how the industry approaches scalability, certificate life-cycles, and post-quantum security.

As always, let me know if you have any feedback, comments or suggestions. Until next time!